

AscensionAI Technical Writeup

Version: 0.4.0 **Document Date:** 2026-05-07 **Author:** Justin Chan **Repository:**
<https://github.com/JustinoChan/AscensionAI>

Abstract

AscensionAI is a reinforcement learning system that trains an autonomous agent to play *Slay the Spire* (Ironclad) end-to-end against the live, modded game process. The system uses a structured 530-dimensional observation encoder, a 134-action discrete action space with legal-action masking, dense reward shaping, behavior cloning warm-start, and Proximal Policy Optimization (PPO) fine-tuning. Training is parallelized across multiple live game instances feeding a central offline trainer over a checkpoint-tagged rollout protocol. The system currently runs on Windows and integrates with ModTheSpire, BaseMod, CommunicationMod, and a bundled SpireComm Python interface. This document describes the architecture, algorithmic choices, implementation, observed throughput, current limitations, and a phased roadmap.

Executive Summary

Aspect	Status
Environment integration	Complete - live STS via CommunicationMod / SpireComm
Observation encoder	Complete - 530-d vector across 11 feature blocks
Action space	Complete - 134 discrete actions, legal-action masking
Reward shaping	Complete - dense per-step + sparse terminal rewards
Behavior cloning	Complete - heuristic demos, supervised cross-entropy, resumable checkpointing
PPO fine-tuning	Complete - clipped policy, GAE, entropy annealing, BC anchor loss, target-KL early stop
Parallel rollout collection	Complete - multi-instance workers with stale-rollout rejection
Offline trainer	Complete - batch-merge, atomic checkpoint save
GUI control panel	Complete - Windows-native launcher and monitor
Win-rate convergence	Not yet demonstrated at scale
Cross-platform support	Windows-only for now

Headline metrics (current state):

- Observation dimensionality: 530
 - Action space size: 134
 - Policy/value network: 530 -> 256 -> 256 -> {134 logits + 1 value}, ~235K parameters
 - Monster knowledge base: 66 monsters x 7 behavioral flags x 8-d identity embedding
 - Typical BC game length: 60-200 transitions
 - Typical BC throughput: 1 game / 30-90 seconds (Fast Mode + Super Fast Mode)
 - Per-step inference: <5 ms on CPU (no GPU required)
-

Table of Contents

1. [Project Basics](#1-project-basics)
 2. [System Architecture](#2-system-architecture)
 3. [Observation Space](#3-observation-space)
 4. [Action Space](#4-action-space)
 5. [Reward Shaping](#5-reward-shaping)
 6. [Behavior Cloning](#6-behavior-cloning)
 7. [PPO Fine-Tuning](#7-ppo-fine-tuning)
 8. [Parallel Training Architecture](#8-parallel-training-architecture)
 9. [Performance and Throughput Estimates](#9-performance-and-throughput-estimates)
 10. [Code Organization](#10-code-organization)
 11. [Reliability and Safety](#11-reliability-and-safety)
 12. [Current Limitations and Known Issues](#12-current-limitations-and-known-issues)
 13. [Roadmap and Future Directions](#13-roadmap-and-future-directions)
 14. [Recommended Workflows](#14-recommended-workflows)
 15. [Metrics Reference](#15-metrics-reference)
 16. [Hyperparameter Reference](#16-hyperparameter-reference)
 17. [Glossary](#17-glossary)
-

1. Project Basics

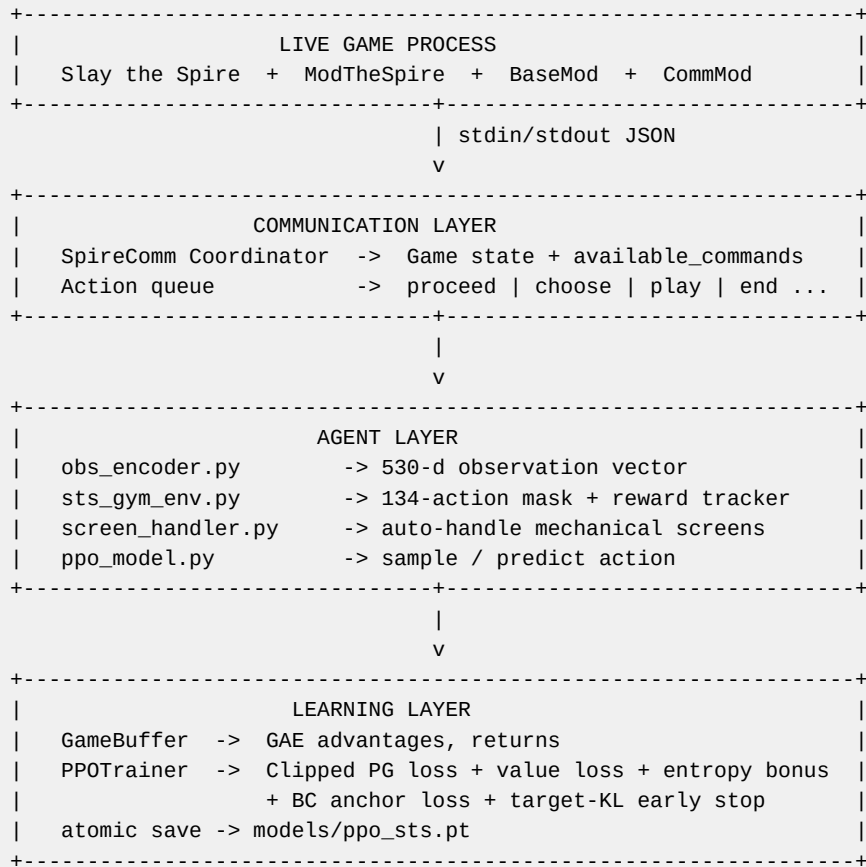
AscensionAI is a reinforcement learning project for training an AI agent to play *Slay the Spire*, currently focused on Ironclad. The project connects to a live *Slay the Spire* process through ModTheSpire, BaseMod, CommunicationMod, and the bundled SpireComm Python interface. The Python side wraps the game in a Gymnasium-style interface with a fixed observation vector, discrete action space, legal-action masking, shaped rewards, and PPO training.

The project is designed around long-running autonomous training rather than one-off scripted play. It supports behavior cloning warm starts, PPO fine-tuning, parallel rollout collection, offline PPO updates, greedy evaluation, passive game logging, training-progress plotting, and a Windows control panel for launching and monitoring multiple game instances.

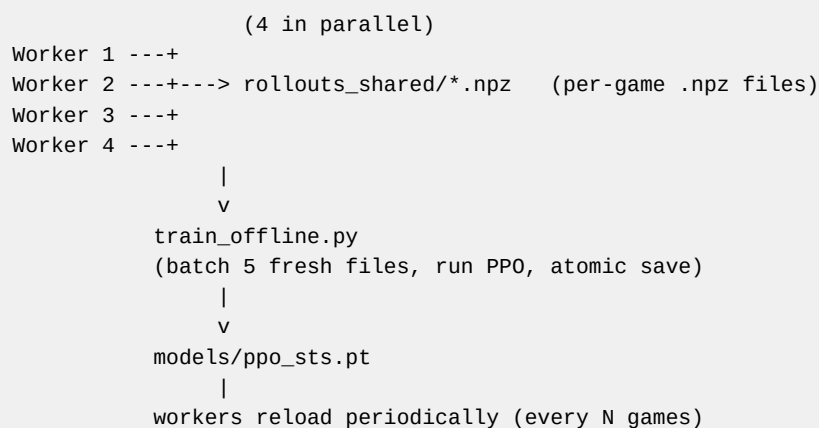
Configuration	Default
Game	Slay the Spire (Mod the Spire)
Character	Ironclad
Ascension level	0
Environment interface	CommunicationMod via SpireComm
RL framework	Custom PPO implementation in PyTorch
Policy	Actor-critic MLP, 2 hidden layers of 256 units, Tanh activation
Warm-start	Behavior cloning from a hand-coded heuristic
Scaling strategy	Multiple rollout workers + central offline trainer
Hardware target	CPU-only (no GPU required)

2. System Architecture

The project is organized around a four-tier pipeline: live game, communication layer, agent layer, and learning layer.



For parallel training the topology fans out:



Critical design decisions:

Decision	Rationale
CPU-only inference	Network is small (~235K params); GPU saturated by other costs (game speed, IPC).
Per-game .npz rollouts	Each rollout is self-contained, atomic, deletable, and carries checkpoint metadata.

Decision	Rationale
Stale-rollout rejection	Workers can lag the trainer; old rollouts harm PPO objective if importance ratios drift too far.
Heuristic + RL hybrid	RL owns "decisions that matter"; heuristics keep the live process from stalling on edge cases.
Action masking (not penalty)	Illegal actions get probability zero in sampling - no wasted gradient steps on impossible plays.

3. Observation Space

Total observation length: **530 floats** (= ~2.1 KB per state).

3.1 Observation block breakdown

Block	Dim	Description
Player state	15	HP / max HP, energy, block, gold, floor, act, in-combat flags
Screen-type one-hot	14	NONE / MAP / EVENT / CHEST / SHOP / REST / ... / HAND_SELECT / GRID / GAME_OVER
Hand cards (10 slots x 16)	160	Per-card: identity, type, cost, upgrade, exhausts, damage/block, playable
Monsters (5 slots x 30)	150	Per-monster: HP/block/intent + 8-d identity embedding + 3 move-history IDs + 7 behavioral flags
Player powers	20	Strength, Dexterity, Vulnerable, Weak, Frail, Ritual, etc.
Monster powers (5 x 8)	40	Per-monster top powers (Strength, Vulnerable, ...)
Choice list features	7	Number/type-distribution of currently offered choices
Relics	25	Relic feature bag (presence + key-effect flags)
Potions (5 x 8)	40	Per-slot: id, target type, cost, presence, value flags
Deck profile	20	Card-type distribution, average cost, curse/status counts, upgrade ratio
Map lookahead	39	4 next choices x (one-hot type + 3-floor BFS density) + 3 globals
Total	530	

3.2 Monster knowledge base

The encoder includes a built-in database of all **66 STS1 monsters**. Each monster contributes:

- Identity embedding (8-d): unique deterministic fingerprint per monster ID (cached, L2-normalized).
- Move history (3-d): last move, second-to-last move, and current intent's move ID.
- Behavioral flags (7-d): binary flags for hidden mechanics that would otherwise take thousands of games to discover.

Flag	Meaning	Example monsters
enrages on skill	Punishes skill cards	Gremlin Nob
splits low hp	Splits at ~50% HP	Slime Boss, Acid Slime L
scales strength	Stacking strength buff	Cultist, Time Eater
multi attacker	Multi-hit attack patterns	Hexaghost, Nemesis, Champ
retaliates	Reactive damage on hit	Spiker, Shelled Parasite
escapes	Can flee combat	Mugger, Looter, all gremlins
spawns_minions	Summons more enemies	Gremlin Leader, Reptomancer, Bronze Automaton

The `spawns_minions` flag is the agent's primary signal for the "focus the spawner" strategy. It is paired with a strong reward bonus (see §5).

3.3 Map lookahead

Rather than encoding only the immediate map choice, the encoder runs a **breadth-first traversal of the next 3 floors** from each candidate node, counting reachable rooms by type (Monster, Elite, Rest, Shop, Unknown, Treasure). This lets the policy reason about path *consequences* (e.g., "this path leads to two elites and no rest") rather than only the next room.

4. Action Space

Total: **134 discrete actions**, with a legal-action mask computed each step.

Action range	Indices	Count	Description
Targeted card play	0-49	50	hand slot (0-9) x monster slot (0-4)
Untargeted card play	50-59	10	hand slot 0-9
End turn	60	1	
Targeted potion	61-85	25	potion slot (0-4) x monster slot (0-4)
Untargeted potion	86-90	5	potion slot 0-4
Choice selection	91-130	40	choice index 0-39 (events, rewards, map, shops)
Proceed	131	1	
Leave / cancel	132	1	
No-op (request state)	133	1	
Total		134	

The legal mask zeros out actions that the current screen does not support. For example, on a CARD_REWARD screen only choice indices and the skip option (proceed) are legal; on a combat turn only playable hand cards (with valid energy) and end-turn are legal.

Why one fixed action space rather than a per-screen variable space: the policy and value heads stay structurally constant, which makes BC and PPO training simpler and lets a single policy cover combat and non-combat decisions.

5. Reward Shaping

Reward shaping is dense to accelerate early learning. All reward sources fire each environment step. Spawner-related shaping is intentionally amplified.

5.1 Reward weights

Source	Magnitude	Notes
Gold gain	+0.01 / gold	
New relic	+1.0	
Card removed from deck	+0.2 / card	(card removal events)
Floor advanced	+0.5	
HP loss	?0.05 / HP	Constant penalty to discourage damage taken
Generic enemy damage dealt	+0.02 / HP	Applied to total enemy HP delta
Generic monster killed	+0.5 / kill	
Spawner damage bonus	+0.08 / HP	On top of generic - total 5x generic for spawners
Spawner kill bonus	+5.0	On top of generic - total 11x generic kill for spawners
Act advanced (act 2+)	+10.0	Encourages full-act progression
Victory	+50.0	Terminal
Defeat	?15.0 + 0.3 x floor	Floor-prorated to avoid instant runs being equally bad

5.2 Rationale

- HP loss penalty is calibrated so a 10-damage hit costs ?0.5, equal to a generic monster kill. This balances offense and defense early in training.
- Spawner shaping is the critical incentive against "minion farming." Without it, killing easy minions can feel as rewarding as killing the spawner per unit time.
- The act-advance bonus is a structural shaping term: it discourages stalling in act 1 and rewards real progression.

5.3 Shaped vs. terminal balance

For a typical successful 16-floor run with 1 victory, the shaped reward sum is roughly:

```
Shaped:   ~30-50   (gold, damage dealt, kills, floor progression)
Terminal: +50      (victory)
Total:    ~80-100
```

For a defeat at floor 5:

```
Shaped:    ~5-10
Terminal:  -15 + 5*0.3 = -13.5
Total:     ~-5 to -10
```

The shaped portion is meaningful but not so dominant that the terminal signal becomes noise.

6. Behavior Cloning

BC is a supervised warm-start phase. A hand-coded heuristic plays full games while the policy network learns to imitate its action distribution.

6.1 Heuristic coverage

The heuristic handles every decision surface:

- Combat: card scoring (block when about to take damage, attack otherwise), spawner targeting (priority over low-HP minions), potion usage thresholds
- Card rewards: type-weighted picks favoring scaling damage and key skills
- Events: routed through pick_event with hand-mapped good/bad choices
- Boss relics, rest sites, shops, map paths
- Mechanical screens (chest open, grid confirm, hand-select fallback)

6.2 Resumable BC checkpointing

Long BC collection (150-200 games) takes 1-3 hours and is fragile to STS crashes. The system saves a per-game checkpoint at `models/ppo_sts_bc_progress.npz` after every completed BC game:

Field	Type
observations	float32 array
actions	int64 array
action masks	bool array
games done	int64
total steps	int64
target games	int64
saved_at	ISO timestamp

If STS crashes at game 145/150, restarting the same mode resumes from the saved demos. The progress file is automatically removed after successful BC training.

6.3 Supervised training loss

```
L_BC = CrossEntropy(policy(obs), heuristic_action_id)
      restricted to legal actions via action mask
```

Default: 30 epochs, batch size 64, learning rate 1e-3, ~200 BC games producing ~30k-60k labeled transitions.

7. PPO Fine-Tuning

PPO fine-tunes the BC policy through online RL. The algorithm follows the standard PPO formulation with several stability additions specific to BC-warm-started agents.

7.1 Loss decomposition

```
L_total = L_PG (clipped) + c_v * L_VF + c_e * H(pi) + c_BC * L_BC_anchor
```

Term	Coefficient	Purpose
L_PG (clipped surrogate)	1.0	Standard PPO policy gradient with clip epsilon = 0.15-0.20
L_VF (value loss)	0.5	MSE between predicted value and discounted return
H(pi) (entropy bonus)	0.05 -> 0.01 (annealed)	Encourages exploration early, exploitation late
L_BC_anchor	0.02	KL-style regularizer pulling toward original BC distribution on demo states

7.2 Stability mechanisms

- Target-KL early stopping: if approximate KL between old and new policy exceeds 0.03, the current update halts. Prevents large policy jumps.
- Entropy annealing: linear decay from 0.05 to 0.01 over the configured PPO game budget (or first 200 games in unlimited mode). Early exploration, late exploitation.
- BC anchor loss: evaluated on a held-out subset of the BC demo set. Anchors the policy near the heuristic distribution and reduces catastrophic forgetting of useful prior behavior.
- Gradient clipping: max norm 0.5.
- GAE returns: gamma = 0.995, lambda = 0.95.
- Per-game PPO batches: updates fire every --games-per-update (default 4) completed games. Larger batches stabilize gradients at the cost of update frequency.

7.3 Target-KL diagnostic example

For a healthy run, approximate KL per epoch should sit roughly in the range:

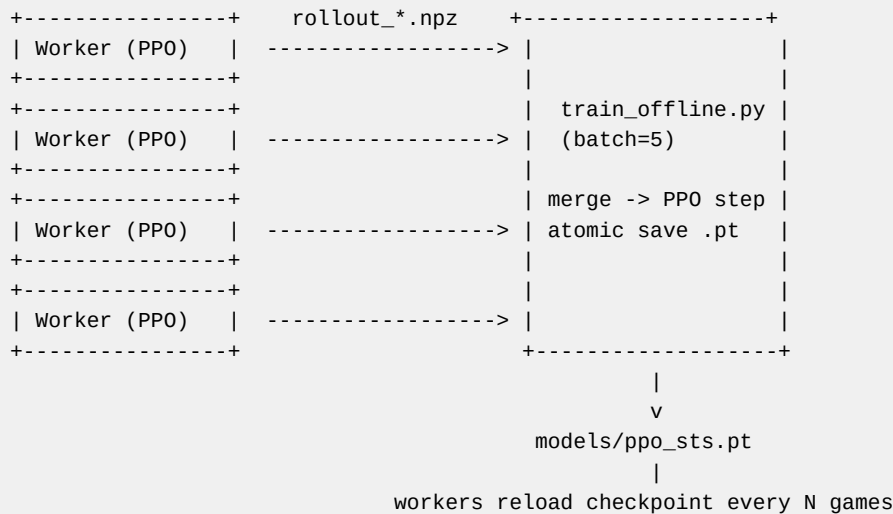
Epoch	ApproxKL	Action
1	0.005-0.015	ok
2	0.010-0.025	ok
3	0.015-0.030	approaching limit
4	>0.030	early-stop kicks in

A consistently early-stopping run indicates the learning rate or entropy is too high for the current data distribution.

8. Parallel Training Architecture

Live-game training is bottlenecked by single-instance simulation speed. The parallel architecture decouples data collection from policy updates.

8.1 Components



8.2 Rollout file contents

Each .npz file represents one completed game:

Field	Type	Description
obs	float32 [T, 530]	Observation history
actions	int64 [T]	Sampled action ids
rewards	float32 [T]	Per-step shaped + terminal
done	bool [T]	Episode termination flags
masks	bool [T, 134]	Legal-action masks at decision time
log_probs	float32 [T]	Old log-probs for importance sampling

Field	Type	Description
values	float32 [T]	Old value estimates
meta.checkpoint_id	int64	Update count of the checkpoint that produced this rollout
meta.game_outcome	dict	Final floor, victory flag, hp, act

8.3 Stale rollout handling

The trainer rejects rollouts that lag too far behind the current checkpoint:

```
if (current_update_count - rollout.checkpoint_id) > MAX_LAG (default: 10):
    reject -> "stale" counter incremented
```

Workers reload the model checkpoint periodically (default: every 5 games) to keep their on-policy assumption approximately valid.

8.4 Throughput multiplier (estimated)

Workers	Games / hour (with Super Fast Mode)	Wall-clock cost
1	~30-60	baseline
2	~60-110	low marginal CPU cost
4	~110-200	comfortable on 8-core / 16GB
6	~140-250	requires 16+ GB RAM, watch for STS instability
8+	diminishing	mod thread contention dominates

9. Performance and Throughput Estimates

Live-game RL fundamentally throttles on game-simulation speed. Numbers below are observed/estimated on an AMD Ryzen 7 9800X3D, 32 GB RAM, with Super Fast Mode at 200% speed.

9.1 Time per game



10. Code Organization

10.1 Top-level

File	Responsibility
AscensionAI.pyw	Windows control panel: launches STS instances, writes CommunicationMod commands, tails logs, detects worker crashes, provides Stop Now / Finish && Stop.
launch_workers.ps1	PowerShell command-line launcher (BC, PPO, eval, logger, parallel modes).
requirements.txt	numpy, torch, gymnasium, psutil, matplotlib.

10.2 Scripts

File	Responsibility
obs_encoder.py	530-d observation construction, monster knowledge base, map lookahead.
sts_gym_env.py	Action space, action masking, flat-id ? SpireComm action conversion, RewardTracker.
ppo_model.py	GameBuffer (GAE), PPOTrainer (clipped policy, value loss, entropy, BC anchor, target-KL early stop, atomic checkpoint save/load).
behavior_clone.py	Heuristic policy + supervised BC training driver. Includes resumable progress checkpointing.
train_bc_ppo.py	End-to-end BC -> PPO pipeline in a single session.
train_ppo.py	Single-instance PPO training.
rollout_worker.py	Worker for parallel rollout collection.
train_offline.py	Offline trainer that consumes rollouts and updates the shared model.
screen_handler.py	Shared mechanical screen handling for non-decision surfaces.
eval_model.py	Greedy (sampling-free) evaluation harness with fixed-seed support.
game_logger.py	Passive game-state recorder for debugging and trace analysis.
fight_tracker.py	Per-fight elite/boss outcome tracking. Handles the in-combat-on-death edge case.
game_data.py	Static card / relic / potion knowledge tables.
make_eval_seeds.py	Deterministic seed list generator for fair comparisons.
analyze_training_rewards.py	Reads training_stats.csv; reports reward-vs-outcome correlations.
analyze_trace.py	Analyzes passive-logger traces for debugging.
plot_training.py	Matplotlib training-curve plot generator.

10.3 External

Path	Description
external/spirecomm/	Bundled SpireComm library; speaks the CommunicationMod stdin/stdout protocol.

11. Reliability and Safety

Long-running modded *Slay the Spire* sessions are fragile. Concrete reliability mechanisms:

Mechanism	Where	What it prevents
Atomic model save (tmp + os.replace)	ppo_model.py, train_offline.py	Half-written .pt files on crash
Atomic rollout write	rollout_worker.py	Half-written .npz files
Per-game BC progress checkpoint	behavior_clone.py, train_bc_ppo.py	Lost demos on STS crash mid-collection
Worker heartbeat files	rollout_worker.py	Silent worker hangs
Worker crash detection + relaunch	AscensionAI.pyw	Stalled multi-instance runs
Orphan Java/Python sweep	AscensionAI.pyw	Leftover processes after Stop Now / Finish && Stop
Stuck-state detection	train_bc_ppo.py, train_ppo.py	Infinite loops on rare game states; dumps to bug_debug.log
HAND_SELECT fallback	behavior_clone.py, screen_handler.py	Headbutt/Warcry-style infinite loops when choice_list is empty
on_error smart recovery	All training scripts	"Invalid command: proceed" loops; auto-falls-back to choose 0
CommunicationMod error logging	All scripts	Silent skipping of failed commands
Stale-rollout rejection	train_offline.py	Off-policy drift from lagging workers

12. Current Limitations and Known Issues

12.1 Scope and characters

- Ironclad only. Card-stat tables and heuristic logic are Ironclad-specific. Other characters would need new card stats and heuristic policies.
- Default Ascension 0. Higher ascensions add elite/normal modifiers and would need additional reward tuning and BC heuristic adjustments.

12.2 Throughput

- Live-game bottleneck. Even at maximum Fast Mode + Super Fast Mode 200%, one game costs ~30-90 seconds. There is no headless simulator integration.
- Single-machine ceiling. ~6-8 concurrent instances is realistic on a 16-core machine; beyond that, mod thread contention dominates. Multi-machine pooling exists but is manual.

12.3 Algorithmic / training risk

- No proven win-rate convergence yet. The full pipeline is in place but has not been run for the full ~3000-game budget needed to demonstrate convergence.
- Reward shaping bias risk. Dense shaping accelerates learning but can encode biases (e.g., over-prioritizing damage dealt vs. healthy block usage). The reward-correlation analyzer exists but tuning is ongoing.
- PPO can forget BC. With high entropy / learning rate / KL movement, PPO can drift away from valuable BC priors. The BC anchor loss mitigates but does not eliminate this.
- Rollout staleness. Workers cache the model and lag the trainer by N games. Beyond ~10 updates lag the importance ratios are stale; rollouts get rejected, throughput drops.

12.4 Coverage gaps

- Shop logic is mostly heuristic. Shop rooms are entered once per floor (a deliberate hack to prevent loops). Real budget-aware shopping should eventually be RL-driven.
- Grid screens are heuristic-handled. Match-and-keep, transform, and special grids have specialized handlers but no learned policy.
- Events with disabled options can be brittle. Most edge cases are handled; new mod-introduced events may not be.

12.5 Platform

- Windows-only at present. GUI, PowerShell launcher, process cleanup, Steam path detection, and CommunicationMod config paths assume Windows. macOS / Linux would need a port.
- Path handling. Several scripts use Windows path separators in defaults; switching shells requires care.

12.6 Evaluation

- Eval game count is the bottleneck for confidence. A 20-game greedy eval has ~22% standard error on win rate; meaningful comparisons need 100-200 games on the same fixed seed list.
- No inter-checkpoint regression detection. If a PPO update degrades performance, it is not flagged automatically; only the rolling-average plot reveals it.

12.7 Known infrequent edge cases

- HAND_SELECT screens for in-combat card selection (Headbutt, Warcry) can have empty choice_list while scr.cards is populated. Patched, but rare new variants of this pattern may surface.
 - Some events (Match and Keep, Falling) produce non-standard grid states; handled via specialized pickers but new modded variants may still hang.
 - ModTheSpire's launch dialog can occasionally need a manual click; handled by a fallback click in the control panel.
-

13. Roadmap and Future Directions

The roadmap is organized into three horizons. Items are listed in rough priority order within each.

13.1 Short-term (next 1-3 months)

1. Run the first full converged training to ground all the estimates above. Target: 200 BC games + 3000 PPO games at 4 workers. Goal: verify avg-100 floor trends upward and victory rate exceeds heuristic baseline.
2. Controlled greedy evaluation pipeline. Run eval_model.py with seeds/eval_200.txt after every 250-500 PPO games. Track win rate, avg floor, elite win rate, boss win rate as time series.
3. Reward-correlation regression check. Use analyze_training_rewards.py to verify shaped reward correlates positively with actual outcomes (final floor, victory). If correlation is weak, retune weights.
4. Checkpoint versioning. Replace single-file ppo_sts.pt with named checkpoints (ppo_sts_g{N}.pt) plus a current.pt symlink. Allow rollback if PPO regresses.
5. Run manifest. Persist hyperparameters, git commit, BC count, PPO count, worker count, entropy schedule, and final-eval results to runs/run_YYYYMMDD/manifest.json.

13.2 Medium-term (3-6 months)

1. Move shop decisions into RL. Build a structured shop observation block (cards on offer with prices, relics on offer, gold available) and let the policy decide buy/skip/remove.
2. Move grid screens into RL. Match-and-keep, transform, and similar grids are good candidates for a learned policy, since the heuristic is already weak there.
3. Headless or accelerated simulator. If a reliable open-source STS simulator becomes available, integrate it as a parallel path. A headless simulator would unlock 10-100x throughput.
4. Better experiment tracking. Integrate with TensorBoard or a lightweight equivalent for live training-curve dashboards.
5. Saved policy-state snapshots for offline inspection. Instead of needing a live eval to inspect top actions, persist a curated set of representative game states and run the policy on them.
6. More aggressive PPO ablations. Sweep learning rate, entropy schedule, BC anchor coefficient, target KL, and games-per-update on a fixed-seed eval set.

13.3 Long-term (6+ months)

1. Additional characters. Silent -> Defect -> Watcher. Each needs new card/relic stats and BC heuristics; the obs encoder schema needs character-specific blocks.
2. Higher ascensions. Ascension 1-20 progression with curriculum-style training.
3. Multi-machine training pool. Replace the manual zip-folder collaboration workflow with a proper rollout server (HTTP or shared filesystem with auth).
4. Cross-platform support. Linux/macOS port for GUI, launcher, and process management.
5. More expressive policy. Replace the 2-layer MLP with a transformer or attention-pooled architecture over the variable-length sub-vectors (hand cards, monsters, choices). Current MLP is fine but capacity-limited.
6. Reward learning. Replace hand-tuned shaping weights with a learned reward model from human play traces or self-play comparisons.

13.4 Risk register

Risk	Likelihood	Impact	Mitigation
BC heuristic ceiling lower than thought	Medium	Medium	Run controlled BC-only baseline eval; compare to PPO checkpoints
PPO catastrophic forgetting	Medium	High	BC anchor loss + target-KL early stop already in place; extend with KL-to-BC penalty if needed
Live-game throughput plateau	High	High	Dependent on headless-simulator availability (medium-term)
Reward shaping bias dominates	Medium	High	Reward-correlation analyzer; long-run validation against pure terminal-only signal
Worker fragility on long runs	Medium	Medium	Heartbeat + relaunch already present; add per-worker memory/CPU watchdog

14. Recommended Workflows

14.1 Fresh-restart workflow

For a clean restart from no useful model:

1. Archive prior artifacts: models/, logs/*.csv, logs/*.log, rollouts_shared/*.npz.
2. Run BC with 150-200 games (Mode: BC -> PPO (End-to-End) in the GUI). Resumable checkpointing protects long runs.
3. (Optional) BC -> PPO sanity: add 0-50 PPO games after BC to validate the pipeline before scaling.

4. Switch to Parallel Workers with 4 workers for the main PPO budget (target ~3000 games).
5. Greedy eval every 250-500 PPO games on seeds/eval_200.txt.
6. Plot trends: `python scripts/plot_training.py --save logs/training_plot.png`. Use rolling avg-100 as the primary signal.

14.2 Hyperparameters: when to change what

Symptom	Likely cause	Adjustment
Avg floor flat for >500 PPO games	Entropy too high, exploration is masking gains	Drop --ent-end from 0.01 -> 0.005
Approximate KL hits target every update	LR too high or entropy too high	Halve --ppo-lr or drop --ent-start
BC anchor loss climbs steadily	Policy drifting from BC distribution	Increase --bc-anchor-coef from 0.02 -> 0.05
Many stale-rollout rejections	Workers reloading too rarely	Decrease worker reload interval
Few wins despite high shaped reward	Reward shaping is misaligned	Run <code>analyze_training_rewards.py</code> ; retune low-correlation weights
Worker crashes / hangs	STS / mod instability	Reduce concurrent workers; enable verbose logging

15. Metrics Reference

Metric	Source	What it measures	Healthy range
Avg-100 final floor	training_stats.csv	Rolling progression depth	Trending upward over 1000+ games
Lifetime avg final floor	training_stats.csv	Long-run baseline	Slowly climbing
Greedy eval avg floor	eval_stats.csv	Sampling-free progression	Should match or exceed avg-100
Greedy eval win rate	eval_stats.csv	Final outcome rate	Higher than heuristic baseline
Elite / boss win rate	fight_stats.csv	Per-fight outcome	Climbing as policy learns
PPO updates	training_stats.csv	Total update count	Monotonic
Trainer-consumed transitions	training_stats.csv	Total samples used	Monotonic
Policy entropy	training_stats.csv	Sampling diversity	0.05 -> 0.01 over annealing
Value loss	training_stats.csv	Critic fitting quality	Decreasing then plateau
Approximate KL	training_stats.csv	Old -> new policy distance	<0.03 (target-KL)
Clip fraction	training_stats.csv	% steps hitting PPO clip	0.1-0.3
Explained variance	training_stats.csv	Critic predictive power	Climbing toward 1.0
BC anchor loss	training_stats.csv	Distance from BC distribution	Stable, not climbing
Stale-rollout rejection count	train_offline_debug.log	Worker-trainer drift	Low (<10% of files)
Reward / final-floor correlation	analyze_training_rewards.py	Shaping validity	>0.4 strongly preferred
Stuck-state dumps	bug_debug.log	Edge-case frequency	Low / decreasing

For *Slay the Spire*, **avg-25 is too noisy** - one early death or one deep run swings it heavily. Avg-100 is the recommended primary signal.

16. Hyperparameter Reference

16.1 BC

Parameter	Default	Notes
--bc-games	50 (suggested 150-200)	Number of heuristic demonstration games
--bc-epochs	30	Supervised training epochs
--bc-lr	1e-3	BC learning rate
--batch-size	64	BC and PPO minibatch size

16.2 PPO

Parameter	Default	Notes
--ppo-games	200	Online RL game budget per session
--ppo-lr	1e-4	PPO learning rate
--gamma	0.995	Discount factor (long-run shaping)
--gae-lambda	0.95	GAE smoothing
--clip	0.15	PPO clip range epsilon
--ent-start	0.05	Initial entropy coefficient
--ent-end	0.01	Final entropy coefficient (annealed)
--target-kl	0.03	Per-update KL early-stop threshold
--bc-anchor-coef	0.02	BC distribution regularizer
--n-epochs	4	PPO epochs per update
--games-per-update	4	PPO updates fire every N completed games
--max-grad-norm	0.5	Gradient clipping

16.3 Network

Parameter	Default
Hidden layers	(256, 256)
Activation	Tanh
Optimizer	Adam
Total parameters	~235,000

17. Glossary

Term	Meaning
BC	Behavior cloning - supervised imitation of a heuristic.
PPO	Proximal Policy Optimization - clipped policy-gradient RL algorithm.
GAE	Generalized Advantage Estimation - variance-reduced advantage estimator.
Spawner	Enemy that summons additional minions during combat (Gremlin Leader, Reptomancer, Bronze Automaton).
CommunicationMod	STS mod that exposes game state and accepts actions over stdin/stdout.
SpireComm	Python library that speaks the CommunicationMod protocol.
Action mask	Per-step boolean vector marking which of the 134 actions are legal.
Stale rollout	A rollout produced by a checkpoint too far behind the current trainer state.
BC anchor loss	A regularization term that pulls the PPO policy toward the original BC distribution on demo states.
Target-KL	A threshold on approximate KL divergence used to early-stop PPO updates that move the policy too far.
Avg-100	Rolling average over the last 100 games - the recommended primary learning-progress signal.

Document History

Version	Date	Notes
0.1	2026-04-15	Initial writeup
0.2	2026-04-26	Added monster knowledge base, parallel architecture
0.3	2026-05-06	Added BC progress checkpointing, refined sections
0.4	2026-05-07	Full restructure with tables, estimates, ASCII charts, expanded limitations and roadmap